

Docker Containers, Images, Internals & Dockerfile Basics

Docker Containers, Images, Internals & Dockerfile Basics

1. Why Do We Need Containers?

Suppose we build a Spring Boot application.

To run it on another machine, we may need to tell someone:

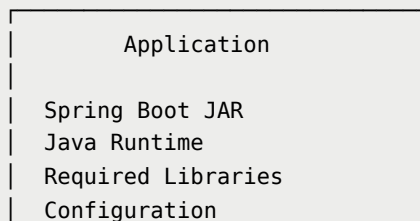
```
Install Java 21
Install the required dependencies
Configure the environment
Set environment variables
Install required libraries
Run the application
```

This creates a common problem:

The application may work correctly on one machine but fail on another because the environments are different.

Instead of recreating the environment manually on every machine, we can package the application together with the environment required to run it.

Conceptually:



Dependencies

This makes application deployment much more predictable.

A container packages an application with the environment required to run it.

2. What Exactly Is Docker?

Containers are built using capabilities provided by the operating system, particularly the Linux kernel.

Linux already provides mechanisms for:

- Process isolation
- Resource limits
- Filesystem isolation
- Network isolation

Docker makes these capabilities much easier to use.

For example:

```
docker run nginx
```

Conceptually, Docker is being asked:

- Take this image
- ↓
- Create an isolated environment from it
- ↓
- Start the application

Docker provides the tooling required to:

- Build images
- Create containers
- Start and stop containers
- Manage networking

Manage storage
Manage container resources

3. Docker Image vs Docker Container

This is one of the most important Docker concepts.

A useful analogy comes from Java.

Suppose we build:

```
app.jar
```

The JAR itself is not the running application.

When we execute:

```
java -jar app.jar
```

a process starts.

Docker follows a similar idea:

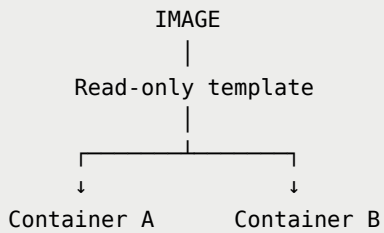
```
Docker Image  
  ↓  
docker run  
  ↓  
Docker Container
```

An **image** is the packaged blueprint.

A **container** is an instance created from that blueprint.

Docker Image

A Docker image is a **read-only template used to create containers**.



Multiple containers can be created from the same image.

A useful programming analogy is:

```
Class  → Object  
Image → Container
```

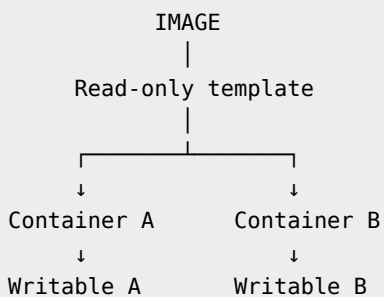
The image describes what should exist.

The container is the actual instance created from that image.

Docker Container

A container is an isolated runtime instance created from an image.

Each container gets its own writable layer.



Changes made inside Container A do not automatically modify Container B or the original image.

4. What Is Inside a Docker Image?

Consider a Spring Boot application.

A Docker image might conceptually contain:

Spring Boot app.jar
Java Runtime
Linux Libraries
Base Filesystem

Once this image is available on a compatible Docker environment:

```
docker run my-app
```

Docker can create a container and start the Spring Boot application.

5. Docker Images Are Immutable

Docker images are generally treated as **immutable artifacts**.

Suppose we have:

```
my-app:v1
```

containing:

```
Java 21  
Spring Boot application v1
```

If we modify the application, we normally build another image:

```
my-app:v2
```

instead of modifying the original image.

This gives us **reproducibility**.

If:

```
my-app:v1
```

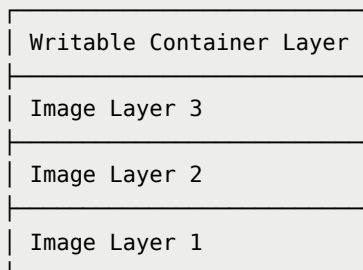
worked previously, the same image can be deployed again.

6. Containers Are Writable

Although images are read-only, a running container needs somewhere to store changes.

Docker adds a **writable container layer** on top of the image.

Container



The underlying image remains unchanged.

Any changes made while the container is running are stored in the container's writable layer.

7. Containers Are Disposable

Containers are normally treated as disposable runtime environments.

If a container crashes or becomes corrupted:

```
Existing Image
  ↓
Create another container
```

Instead of manually repairing the same container forever, we can create a fresh one from the image.

This is an important shift from traditional server management.

8. What About Persistent Data?

Consider MySQL running inside a container.

The database stores:

```
Users
Orders
Payments
```

If all of this data exists only inside the container's writable layer:

```
Container deleted
  ↓
Data deleted
```

That is not acceptable for persistent data.

Persistent data should normally live outside the disposable container filesystem using mechanisms such as:

```
Docker Volumes
```

A useful mental model is:

```
Container = Disposable
```

```
Data = Persistent
```

Volumes can therefore survive even when containers are recreated.

9. The Complete Docker Flow

```
Dockerfile
  ↓
docker build
  ↓
Docker Image
  ↓
docker run
  ↓
Docker Container
  ↓
Application Process
```

This flow is fundamental to understanding Docker.

10. A Container Is Fundamentally a Process

When we run:

```
java -jar app.jar
```

Linux creates a process.

For example:

```
PID 4521
java -jar app.jar
```

Docker does not fundamentally create a tiny virtual machine every time a container starts.

If we run:

```
docker run nginx
```

Linux ultimately starts an `nginx` process.

The difference is that this process is placed inside isolation mechanisms.

A useful mental model is:

```
Container
  =
Linux Process
  +
Isolation
  +
Resource Limits
  +
Layered Filesystem
```

Docker asks Linux to:

```
Start this process

But...

Isolate what it can see
Control what resources it can use
Give it its own filesystem view
Give it its own network environment
```

This is why containers are much lighter than traditional virtual machines.

11. Practical Demo: Image vs Container

The goal of this demo is to prove that:

```
hello-demo image
hello = "Hello"
  /      \
 /        \
```

container-a	container-b
"Hello from A"	"Hello"

Both containers start from the same image, but each gets its own writable layer.

Enter the Ubuntu VM

If using Multipass:

```
multipass list
```

Enter the VM:

```
multipass shell devops
```

Check Docker:

```
docker --version
```

If Docker requires elevated privileges, use:

```
sudo docker . . .
```

Create the Demo Directory

```
mkdir docker-demo  
cd docker-demo
```

Create a Dockerfile:

```
nano Dockerfile
```

Add:

```
FROM alpine

RUN mkdir /app
RUN echo "Hello" > /app/hello.txt

CMD ["sh"]
```

This Dockerfile creates an image containing:

```
/app/hello.txt
```

with:

```
Hello
```

Build the Image

```
docker build -t hello-demo .
```

Here:

```
-t
```

means **tag**.

It assigns the name:

```
hello-demo
```

to the image.

The:

```
.
```

means:

```
current directory
```

Docker uses the current directory as the **build context**.

The flow is:

```
Dockerfile
  ↓
docker build
  ↓
hello-demo image
```

Check available images:

```
docker images
```

Example:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
hello-demo	latest	abc123...	...	...
alpine	latest	xyz456...	...	...

At this point:

```
Image exists
Container does not
```

Nothing is running yet.

Create Container A

```
docker run -it --name container-a hello-demo
```

- `i` means:

```
interactive
```

It keeps standard input open.

- `t` means:

```
allocate a terminal
```

Together:

```
-it
```

means:

| Run the container interactively through a terminal.

- `--name container-a` assigns a readable name to the container.

Inside the container, check:

```
cat /app/hello.txt
```

Output:

```
Hello
```

Now modify the file:

```
echo "Hello from Container A" > /app/hello.txt
```

Check again:

```
cat /app/hello.txt
```

Output:

```
Hello from Container A
```

Exit:

```
exit
```

Container A now has its own filesystem change.

The original image is still unchanged.

Create Container B From the Same Image

```
docker run -it --name container-b hello-demo
```

Check:

```
cat /app/hello.txt
```

Output:

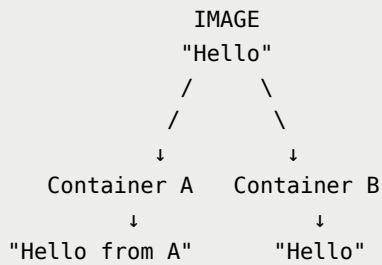
```
Hello
```

Container B does **not** see:

```
Hello from Container A
```

because that change exists only in Container A's writable layer.

This proves:



The image remains unchanged.

View All Containers

```
docker ps -a
```

Example:

CONTAINER ID	IMAGE	NAMES
xxxxxx	hello-demo	container-b
yyyyyy	hello-demo	container-a

`docker ps` shows running containers.

`docker ps -a` shows running and stopped containers.

12. How Containers Work Internally

A container relies heavily on Linux kernel features.

The three key concepts are:

```
Namespaces
  ↓
What can the process see?

cgroups
  ↓
How many resources can it use?

Layered Filesystem
```

↓
What filesystem does it see and modify?

13. Linux Namespaces

Namespaces provide **isolation of view**.

A simple mental model is:

Namespace = What can this process see?

Imagine two people inside the same building.

Person A → Room A

Person B → Room B

They are inside the same building, but each person sees a different environment.

Containers work in a similar way.

Processes still share the same Linux kernel, but namespaces give them isolated views of different system resources.

14. PID Namespace

Suppose the host sees:

Host OS

PID 1	systemd
PID 500	Chrome
PID 800	Docker
PID 1500	nginx

The host may see the container's nginx process as:

```
PID 1500 → nginx
```

Inside the container:

```
ps
```

may show:

```
PID 1 → nginx
```

Both are correct.

HOST

```
PID 1    systemd  
PID 500  chrome  
PID 1500 nginx
```

```
↑  
|  
CONTAINER
```

```
PID 1 nginx
```

The **PID namespace** gives the container its own view of processes.

The container normally cannot see every process running on the host.

15. Network Namespace

Containers can also have isolated network environments.

Suppose:

```
Container A → Application on port 8080  
Container B → Application on port 8080
```

Both applications can use port `8080` because each container has its own network namespace.

Each can have its own:

```
Network interfaces
IP addresses
Ports
Routing
```

Docker can then map different host ports.

Example:

```
docker run -p 8081:8080 app
docker run -p 8082:8080 app
```

Conceptually:

```
Host :8081
  ↓
Container A :8080

Host :8082
  ↓
Container B :8080
```

Inside both containers, the application still listens on:

```
8080
```

16. Mount Namespace

The host may contain:

```
/home  
/etc  
/var  
...
```

We usually do not want a container to see the entire host filesystem.

The **mount namespace** gives the process an isolated filesystem view.

A container may see:

```
/
├─ app
├─ bin
├─ etc
├─ lib
└─ tmp
```

The container believes this is:

```
/
```

but it is not necessarily the host's root filesystem.

It is the container's isolated filesystem view.

17. Namespaces Do Not Control Resource Usage

Namespaces answer:

```
What can this process see?
```

But suppose a container starts consuming:

```
100% CPU
```

or:

```
20 GB RAM
```

Namespaces do not solve that problem.

We need another mechanism:

```
cgroups
```

18. cgroups

`cgroups` stands for:

```
Control Groups
```

They answer:

```
How much can this process use?
```

Suppose a server has:

```
8 CPU cores  
16 GB RAM
```

and runs:

```
Container A  
Container B  
Container C
```

Without resource controls, one badly behaving application could consume a large amount of CPU or memory and affect the other applications.

With cgroups, resource limits can be applied.

For example:

```
Container A
Memory = 512 MB

Container B
Memory = 1 GB

Container C
Memory = 2 GB
```

Example Docker command:

```
docker run --memory=512m my-app
```

The container's processes can then be placed in a control group with a 512 MB memory limit.

Why cgroups Matter in DevOps

Imagine one machine is running:

```
User Service
Payment Service
Notification Service
Recommendation Service
```

A memory leak in Recommendation Service should not be allowed to consume all available memory and destabilize the other services.

With resource controls:

```
Recommendation Service
  ↓
  cgroup
  ↓
Memory limit: 1 GB
CPU usage: controlled
```

So:

```
Namespaces → Isolation
```

```
cgroups → Resource control
```

19. Container Filesystems

Namespaces can isolate the filesystem view, but the container still needs actual files.

An application may need:

```
/bin  
/etc  
/lib  
/usr  
/app/app.jar
```

Copying an entire operating-system filesystem every time we create a container would be inefficient.

Docker solves this using **layered filesystems**.

Images are built from reusable layers.

Containers then add a small writable layer on top.

20. Dockerfile Basics

A Docker image needs a repeatable definition.

Suppose we have:

```
app.jar
```

To run it manually we might need to:

```
Install Java 21
Create /app
Copy app.jar
Run java -jar app.jar
```

Instead of manually repeating these instructions, we describe them inside a:

Dockerfile

A Dockerfile is a **recipe for building a Docker image**.

The complete relationship is:

```
Source Code
+
Dockerfile
|
↓
docker build
|
↓
Docker Image
|
↓
docker run
|
↓
Docker Container
```

21. Basic Spring Boot Dockerfile

Suppose the application is already built as:

app.jar

A simple Dockerfile could be:

```
FROM eclipse-temurin:21-jre

WORKDIR /app

COPY app.jar app.jar

CMD ["java", "-jar", "app.jar"]
```

22. FROM

```
FROM eclipse-temurin:21-jre
```

FROM defines the starting image.

Our Spring Boot application requires Java.

Instead of manually installing Java, we start from an existing image containing a Java runtime.

Conceptually:

```
Existing Java Image
    ↓
Add our application
    ↓
Our Docker Image
```

Other examples include:

```
FROM ubuntu:24.04
```

```
FROM node:24
```

```
FROM python:3.13
```

For an already-built Spring Boot JAR:

```
FROM eclipse-temurin:21-jre
```

is a suitable runtime starting point.

23. WORKDIR

```
WORKDIR /app
```

This establishes:

```
/app
```

as the working directory.

It is conceptually similar to:

```
cd /app
```

Later instructions can operate relative to this directory.

Example:

```
/
├── bin
├── etc
└── app
```

24. COPY

```
COPY app.jar app.jar
```

The first:

```
app.jar
```

refers to the file from the Docker build context.

The second:

```
app.jar
```

is the destination inside the image.

Because the working directory is:

```
/app
```

the result becomes:

```
/app/app.jar
```

We could also write:

```
COPY app.jar /app/app.jar
```

25. **RUN**

RUN executes a command while the image is being built.

Example:

```
RUN apt-get update
```

or:

```
RUN mkdir /app
```

Think:

```
RUN
↓
docker build
```

RUN normally modifies the image during the build process.

26. **CMD**

```
CMD ["java", "-jar", "app.jar"]
```

CMD specifies the default command to execute when a container starts from the image.

When we run:

```
docker run my-app
```

Docker eventually executes:

```
java -jar app.jar
```

The important distinction is:

```
RUN
↓
Executed during docker build

CMD
↓
Executed when docker run starts a container
```

27. **ENV**

Environment variables can be defined using:

```
ENV APP_ENV=production
```

Processes inside the container can then access:

```
APP_ENV
```

However, sensitive values such as:

```
Database passwords  
API keys  
Tokens
```

should normally not be baked into the Docker image.

Sensitive configuration should be provided at runtime using an appropriate secret-management mechanism.

28. **EXPOSE**

EXPOSE documents which port the application is expected to listen on.

Example:

```
EXPOSE 8080
```

For a Spring Boot application:

```
Application listens on → 8080
```

EXPOSE by itself does not necessarily publish the port to the host.

Port publishing is normally done when starting the container:

```
docker run -p 8080:8080 my-app
```

29. Important Dockerfile Instructions

```
FROM
Starting image

WORKDIR
Set working directory

COPY
Copy files into image

RUN
Execute commands while building the image

CMD
Default command executed when the container starts

EXPOSE
Document intended application port

ENV
Define environment variables
```

30. Docker Image Layers

Consider a Spring Boot application requiring:

```
Base filesystem
+
Java Runtime
+
Application
```

A naive system might package everything into one giant image.

Suppose:

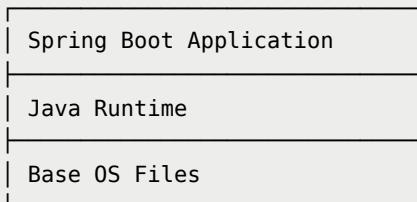
Image = 400 MB

Now we make a tiny application change.

It would be wasteful to rebuild and store another completely independent 400 MB image.

Docker therefore organizes images into **layers**.

Conceptually:



If only the application changes:

```
Base OS      → Same
Java Runtime → Same
Application  → Changed
```

Docker can reuse the unchanged layers.

31. Docker Image = Stack of Layers

Consider:

```
FROM ubuntu:24.04
```

```
RUN apt-get update
```

```
RUN apt-get install -y curl
```

```
COPY app.jar /app/app.jar
```

```
CMD ["java", "-jar", "/app/app.jar"]
```

Conceptually:

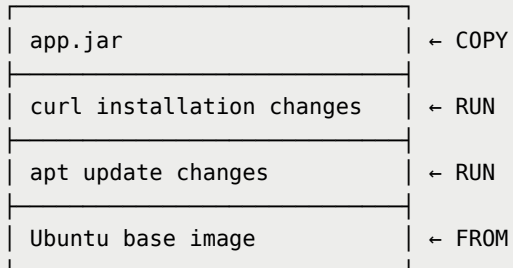
```
FROM ubuntu
  ↓
Base Layer

RUN apt-get update
  ↓
Filesystem changes

RUN apt-get install curl
  ↓
Filesystem changes

COPY app.jar
  ↓
Application files
```

The resulting image can be visualized as:



The container does not normally see directories called:

```
layer1
layer2
layer3
```

It sees one combined filesystem:

```
/
├── bin
├── etc
├── usr
└── app
    └── app.jar
```

The layer structure is primarily important for storage, reuse, distribution, and build optimization.

32. Layers Can Be Shared

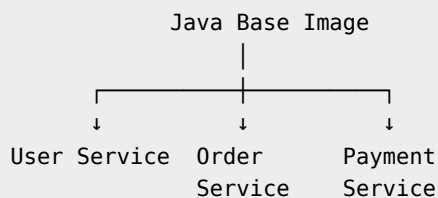
Imagine three Spring Boot services:

```
user-service
order-service
payment-service
```

All of them use:

```
FROM eclipse-temurin:21-jre
```

Conceptually:



The same underlying base layers can be reused instead of storing completely independent copies for every image.

33. Docker Build Cache

Suppose we have:

```
FROM eclipse-temurin:21-jre

WORKDIR /app

COPY app.jar app.jar

CMD ["java", "-jar", "app.jar"]
```

Build the image:

```
docker build -t my-app .
```

During the first build, Docker must process the instructions.

On another build:

```
docker build -t my-app .
```

Docker checks whether it already has a valid cached result for each build step.

If the input to a step has not changed:

```
Reuse cache
```

This can make repeated Docker builds significantly faster.

34. Why Dockerfile Order Matters

Consider:

```
FROM node:24

WORKDIR /app
```

```
COPY . .  
  
RUN npm install  
  
CMD ["npm", "start"]
```

The instruction:

```
COPY . .
```

copies the project into the image.

Now imagine only:

```
src/index.js
```

changes.

The input to:

```
COPY . .
```

has changed.

Docker may therefore have to process later steps again, including:

```
RUN npm install
```

But the dependencies did not actually change.

This wastes time.

35. Better Dependency Caching

A better Dockerfile is:

```
FROM node:24

WORKDIR /app

COPY package.json package-lock.json ./

RUN npm ci

COPY . .

CMD ["npm", "start"]
```

Now if only:

```
src/app.js
```

changes:

FROM	→ Cached
WORKDIR	→ Cached
COPY package*.json	→ Cached
RUN npm ci	→ Cached
COPY application source	→ Changed

The dependency installation can remain cached.

A useful rule is:

```
Things that change rarely
    ↓
Things that change occasionally
    ↓
Things that change frequently
```

Place relatively stable instructions earlier in the Dockerfile when possible.

36. Image Layers vs Container Writable Layer

These concepts should not be confused.

During build time:

```
Read-only Image Layer 3
Read-only Image Layer 2
Read-only Image Layer 1
```

When a container starts:

```
Container Writable Layer
-----
Read-only Image Layer 3
Read-only Image Layer 2
Read-only Image Layer 1
```

Another container gets a different writable layer:

```
Container B Writable Layer
-----
Same underlying image
```

Therefore:

```
BUILD TIME
  ↓
Image layers

RUN TIME
  ↓
Container writable layer
```

37. Dockerfile Best Practices

Understanding image layers makes Dockerfile best practices easier to understand. They are not arbitrary rules. Most exist to improve:

```
Build speed
Image size
Security
Reproducibility
Maintainability
```

Order Instructions by Change Frequency

Avoid:

```
COPY . .

RUN npm install
```

when dependency files can be copied separately.

Prefer:

```
COPY package.json package-lock.json ./

RUN npm ci

COPY . .
```

This allows Docker to reuse expensive dependency installation steps when application code changes but dependencies do not.

38. Understand the Docker Build Context

When we execute:

```
docker build .
```

the:

```
.
```

represents the **build context**.

```
docker build .  
           ↑  
    Current directory
```

Docker's builder can access files in this context for instructions such as `COPY`.

Suppose the project contains:

```
project/  
├─ src/  
├─ target/  
├─ .git/  
├─ node_modules/  
├─ screenshots/  
├─ videos/  
├─ notes/  
└─ Dockerfile
```

Many of these files may not be required while building the image.

39. `.dockerignore`

A `.dockerignore` file prevents unnecessary files from being included in the Docker build context.

Example:

```
.git  
.idea  
*.log  
node_modules  
README.md  
docs
```

The exact contents depend on the project.

Benefits include:

```
Smaller build context
Faster builds
Fewer unnecessary files
Reduced chance of accidentally including sensitive information
```

40. Never Bake Secrets Into Docker Images

Avoid storing secrets directly inside Dockerfiles.

For example:

```
ENV DB_PASSWORD=mySuperSecretPassword
```

Docker images are distributable artifacts.

Users with access to the image may be able to inspect its configuration, metadata, history, or filesystem contents.

Sensitive values should instead be supplied using an appropriate runtime mechanism.

A simple local example:

```
docker run \
  -e DB_PASSWORD=somevalue \
  my-app
```

Production environments may use dedicated secret-management solutions provided by platforms such as Kubernetes, cloud providers, or systems such as HashiCorp Vault.

41. Do Not Install Tools You Do Not Need

Suppose the production application only requires:

```
Java Runtime
+
app.jar
```

The final production image usually does not also need development tools such as:

```
vim
gcc
git
Maven
Node.js
Python
```

Every unnecessary package can increase:

```
Image size
Download time
Attack surface
Vulnerability surface
Maintenance requirements
```

A good runtime image contains only what the application actually needs.

42. Core Mental Models

Image vs Container

```
Image
=
Read-only blueprint

Container
=
Runtime instance created from the image
```

Container Internals

```
Container
=
Process
+
Namespaces
+
cgroups
+
Layered filesystem
```

Linux Isolation

```
Namespaces
=
What can I see?

cgroups
=
How much can I use?
```

Image and Container Storage

```
Image
=
Read-only layers

Container
=
Image layers
+
Writable layer
```

Persistent Data

```
Container
=
Disposable

Persistent Data
```

=
Volumes / External Storage

Docker Workflow

```
Dockerfile
  ↓
docker build
  ↓
Docker Image
  ↓
docker run
  ↓
Docker Container
  ↓
Application Process
```

43. Important Commands From This Lecture

```
docker --version
```

Check the installed Docker version.

```
docker build -t hello-demo .
```

Build an image from the current directory and tag it as `hello-demo`.

```
docker images
```

List Docker images.

```
docker run -it --name container-a hello-demo
```

Create and run an interactive container named `container-a`.

```
docker ps
```

Show running containers.

```
docker ps -a
```

Show running and stopped containers.

```
docker run --memory=512m my-app
```

Start a container with a memory limit.

```
docker run -p 8081:8080 my-app
```

Map host port `8081` to container port `8080`.

Final Takeaway

Docker becomes much easier once we stop thinking of a container as a tiny virtual machine.

At its core:

```
Docker Image
=
Packaged application environment

Docker Container
=
Isolated process created from that image
```

Linux namespaces control what the process can **see**.

cgroups control how many resources the process can **use**.

Docker image layers make application packaging efficient and reusable.

The container gets its own writable layer while the original image remains unchanged.

And the complete lifecycle remains:

```
Write Dockerfile
  ↓
Build Image
  ↓
Run Container
  ↓
Application runs as an isolated process
```